양자컴퓨팅 이론&실습 4주 부트캠프

2일차: 다중 큐비트와 양자 얽힘(Entanglement)

강사: 서상협 ((주)큐렌스)

2026. 11. 18 (수)

2일차 핵심 질문: 독립된 계에서 상호작용 계로

"큐비트가 결합하면 상태공간의 차원은 왜 지수적으로 팽창(2^n)하며, 독립된 상태로 분리할 수 없는 '얽힘'은 어떻게 만들어지는가?"

2일차 학습의 관점: 독립된 상태에서 '비분리 얽힘 상태'로

- 고전 컴퓨터의 50비트는 독립된 2진 스위치 50개에 불과합니다.
- 큐비트가 결합하면 상태를 기술하는 힐베르트 공간의 차원이 2^n**으로 지수적 팽창**을 합니다.
- 두 큐비트가 상호작용하여 개별 상태의 텐서곱으로 분해할 수 없는 **'얽힘 (Entanglement)'** 상태가 탄생합니다.
- 텐서곱($\otimes$)과 2큐비트 유니터리 연산(CNOT)의 선형대수학적 구조를 손계산과 NumPy로 확인합니다.

2일차 세부 운영안

구간	주제	시간	학습자 인지 궤적 및 활동
1부	**다중 큐비트 이론 및 손계산 집중**	75분	2^n 차원 폭발 → 텐서곱($\otimes$) 대수 → CNOT 게이트 → **[손계산 5단계] Bell 회로 유도 및 얽힘 증명**
휴식	**휴식 및 실습 환경 점검**	20분	재충전, 1부 손계산 개별 질의응답, Python/NumPy 실습 환경 점검
2부	**NumPy 다중 큐비트 인터랙티브 실습**	75분	No-Signaling → **[Lab 1–3] 텐서곱 · CNOT · 얽힘 판별식** → **[2부 챌린지] 4대 Bell 상태 합성** → 부분 측정 붕괴

1일차 복습: 단일 큐비트에서 다중 큐비트로의 확장

1일차에 배운 단일 큐비트 시스템 ($\mathbb{C}^2$)

- 2차원 복소 힐베르트 공간: $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ ($|\alpha|^2 + |\beta|^2 = 1$)
- 1큐비트 게이트: 2×2 유니터리 행렬 (X, Y, Z, H, S, T)
- 상태의 기하학: 블로흐 구(Bloch Sphere) 위의 한 점 (θ, ϕ)

오늘 마주할 2큐비트 이상의 복합 시스템 ($\mathbb{C}^2 \otimes \mathbb{C}^2 \cong \mathbb{C}^4$)

- 큐비트가 2개로 늘어나면 기저 상태가 4개(‘00‘, ‘01‘, ‘10‘, ‘11‘)로 늘어납니다.
- 더 이상 블로흐 구 2개만으로는 두 큐비트의 상호작용과 얽힘을 표현할 수 없습니다!
- 두 큐비트가 결합하는 수학적 연산자, **텐서곱(Tensor Product, $\otimes$)**이 필요한 이유입니다.

상태공간 차원의 차이: 고전 비트 50개 vs 큐비트 50개

고전 비트 50개의 상태

- 각 비트는 0 또는 1의 확정된 값 하나를 가집니다.
- 예: '11010010101...' $\Longrightarrow$ 50개의 비트, 즉 **약 6.25 바이트**에 불과합니다.
- 최신 컴퓨터 메모리(수십 GB)와 비교해도 극소량의 데이터에 해당합니다.

큐비트 50개의 상태

- 큐비트 50개는 50개의 숫자로 표현되지 않습니다!
- 50개의 큐비트가 동시에 가질 수 있는 모든 2진수 경우의 수($00\ldots0$부터 $11\ldots1$까지) 마다 **각각 고유한 복소수 진폭**이 붙습니다.
- 필요한 복소수 개수: $2^{50} \approx 1,125,899,906,842,624$**개 (약 1,125조 개!)**

차원의 폭발: 왜 슈퍼컴퓨터의 RAM이 터지는가?

큐비트 수 (n)	필요한 진폭 개수 (2^n)	필요한 메모리 용량 (복소수 16바이트 기준)
1개	$2^1 = 2$개	32 바이트 (스마트워치 수준)
2개	$2^2 = 4$개	64 바이트
10개	$2^{10} = 1,024$개	16 킬로바이트 (KB)
20개	$2^{20} \approx 100$만개	16 메가바이트 (MB, 노트북 가능)
30개	$2^{30} \approx 10$억개	16 기가바이트 (GB, 고성능 워크스테이션 한계)
50개	$2^{50} \approx 1,125$**조 개**	**16 페타바이트 (PB, 전 세계 최고 슈퍼컴퓨터 메모리 초과!)**

결론

양자컴퓨터가 50큐비트만 도달해도 슈퍼컴퓨터가 흉내 낼 수 없는 이유는,
"이 지수적인 2^n 차원 공간의 복소수들을 슈퍼컴퓨터 메모리에 담을 수조차 없기 때문"
입니다.

왜 양자 시뮬레이션은 고전 컴퓨터에서 불가능한가?

선형 대수학적 병목 (Linear Algebra Bottleneck)

- 고전 알고리즘에서 변수가 1개 늘어나면 배열 크기는 $+1$ 됩니다 (선형 증가).
- 양자 시스템에서는 큐비트가 1개 늘어날 때마다 상태벡터 길이가 $\times 2$ **(2배)**가 됩니다.
- 50큐비트 게이트 연산은 $2^{50} \times 2^{50}$ **크기의 복소 행렬**을 곱하는 것과 같습니다.

리처드 파인만(Richard Feynman)의 1981년 통찰

"Nature isn't classical, dammit, and if you want to make a simulation of nature, you'd better make it quantum mechanical!"

자연계의 양자 현상을 컴퓨터로 모사하려면, 컴퓨터 자체가 양자역학적으로 동작해야 합니다.

2큐비트 공간의 4대 기준점 (계산 기저)

큐비트가 2개가 되면, 고전 비트의 4가지 조합(‘00‘, ‘01‘, ‘10‘, ‘11‘)이 기저 벡터가 됩니다:

4차원 열벡터 표현

$$|00\rangle = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}, \quad |01\rangle = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}, \quad |10\rangle = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}, \quad |11\rangle = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}$$

임의의 2큐비트 중첩 상태

$$|\psi\rangle = c_{00}|00\rangle + c_{01}|01\rangle + c_{10}|10\rangle + c_{11}|11\rangle = \begin{pmatrix} c_{00} \\ c_{01} \\ c_{10} \\ c_{11} \end{pmatrix}$$

전체 확률 합 1 조건: $|c_{00}|^2 + |c_{01}|^2 + |c_{10}|^2 + |c_{11}|^2 = 1$

[기정사실화 안내] 텐서곱($\otimes$)의 대수적 정의

수학 기호 $\otimes$에 겁먹지 마세요!

- 교재나 논문에 자주 등장하는 $|0\rangle \otimes |0\rangle = |00\rangle$ 기호는 **'텐서곱(Tensor Product)'**이라 부릅니다.
- 고등 수학에서는 이를 복잡한 텐서 대수로 정의하지만, 양자컴퓨팅에서는 아주 단순합니다:

$$\begin{pmatrix} a \\ b \end{pmatrix} \otimes \begin{pmatrix} c \\ d \end{pmatrix} = \begin{pmatrix} a \cdot \begin{pmatrix} c \\ d \end{pmatrix} \\ b \cdot \begin{pmatrix} c \\ d \end{pmatrix} \end{pmatrix} = \begin{pmatrix} ac \\ ad \\ bc \\ bd \end{pmatrix}$$

- **규칙**: "앞 벡터의 각 성분에 뒤 벡터 전체를 곱해서 세로로 길게 이어 붙이는 블록 쌓기!"

텐서곱 상세 전개 예제: 2차원 벡터의 결합

두 단일 큐비트 상태 $|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$과 $|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$의 텐서곱을 전개해 봅시다:

$|01\rangle = |0\rangle \otimes |1\rangle$의 단계별 계산

$$|0\rangle \otimes |1\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \otimes \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \cdot \begin{pmatrix} 0 \\ 1 \end{pmatrix} \\ 0 \cdot \begin{pmatrix} 0 \\ 1 \end{pmatrix} \end{pmatrix} = \begin{pmatrix} 1 \cdot 0 \\ 1 \cdot 1 \\ 0 \cdot 0 \\ 0 \cdot 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} = |01\rangle$$

$|10\rangle = |1\rangle \otimes |0\rangle$의 단계별 계산

$$|1\rangle \otimes |0\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \otimes \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \cdot \begin{pmatrix} 1 \\ 0 \end{pmatrix} \\ 1 \cdot \begin{pmatrix} 1 \\ 0 \end{pmatrix} \end{pmatrix} = \begin{pmatrix} 0 \cdot 1 \\ 0 \cdot 0 \\ 1 \cdot 1 \\ 1 \cdot 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} = |10\rangle$$

두 벡터의 순서가 바뀌면 결과 벡터의 1이 위치하는 인덱스도 완전히 달라집니다!

행렬의 텐서곱: 2큐비트 유니터리 연산자 구성

단일 큐비트 게이트 2개를 각각의 큐비트에 동시에 적용할 때, 전체 연산자 행렬은 **게이트 행렬들의 텐서곱(4×4)**으로 만들어집니다:

$A \otimes B$의 블록 행렬 정의

$$A \otimes B = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \otimes B = \begin{pmatrix} a_{11}B & a_{12}B \\ a_{21}B & a_{22}B \end{pmatrix}$$

예시: 1번 큐비트에만 H를 적용하고 2번 큐비트는 가만히 둘 때 ($H \otimes I$)

$$H \otimes I = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \otimes \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \cdot I & 1 \cdot I \\ 1 \cdot I & -1 \cdot I \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \end{pmatrix}$$

텐서곱의 핵심 성질: 혼합곱 공식(Mixed-Product Property)

양자 회로 계산을 비약적으로 단순화해 주는 가장 중요한 대수적 정리입니다:

혼합곱 공식 (Mixed-Product Property)

$$(A \otimes B)(C \otimes D) = (AC) \otimes (BD)$$

"텐서곱을 한 뒤 행렬을 곱하는 것은, 각 게이트끼리 먼저 곱한 뒤 텐서곱을 취하는 것과 같다!"

행렬과 벡터의 결합 형태

$$(A \otimes B)(|\psi_1\rangle \otimes |\psi_2\rangle) = (A|\psi_1\rangle) \otimes (B|\psi_2\rangle)$$

- 4×4 행렬 전체를 거대하게 계산하지 않고도,
- 1번 큐비트 연산 $A|\psi_1\rangle$과 2번 큐비트 연산 $B|\psi_2\rangle$를 각각 독립적으로 계산한 뒤 텐서곱할 수 있습니다!

서로 대화하는 2큐비트 게이트: CNOT (제어-NOT)

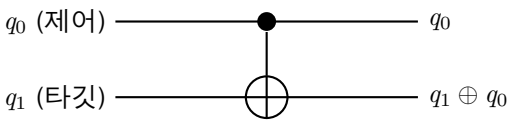
1큐비트 게이트(H, X, Z)는 각 큐비트가 혼자 도는 연산이었습니다. 양자컴퓨터가 강력해지려면 **두 큐비트가 서로 영향을 주고받는 제어 연산**이 필수적입니다.

CNOT (Controlled-NOT) 게이트의 동작 규칙

- 1번 큐비트(**제어 큐비트**): 상태를 바꾸지 않고 신호만 줍니다.
- 2번 큐비트(**타깃 큐비트**): 제어 큐비트의 상태에 따라 동작합니다.

"제어 큐비트가 1이면, 타깃 큐비트를 NOT(뒤집기) 한다!"

CNOT 게이트의 진리표와 회로 기호



입력 상태 ($\lvert q_0\, q_1\rangle$)	제어 비트 (q_0)	출력 상태
$\lvert 00\rangle$	0 (뒤집지 않음)	$\lvert 00\rangle$
$\lvert 01\rangle$	0 (뒤집지 않음)	$\lvert 01\rangle$
$\lvert 10\rangle$	1 (뒤집음: $0 \to 1$)	$\lvert 11\rangle$
$\lvert 11\rangle$	1 (뒤집음: $1 \to 0$)	$\lvert 10\rangle$

CNOT의 4×4 행렬 구조 유도

4가지 계산 기저에 작용한 출력을 열벡터로 나열하면 CNOT 행렬이 완성됩니다:

$$CX|00\rangle = |00\rangle, \quad CX|01\rangle = |01\rangle, \quad CX|10\rangle = |11\rangle, \quad CX|11\rangle = |10\rangle$$

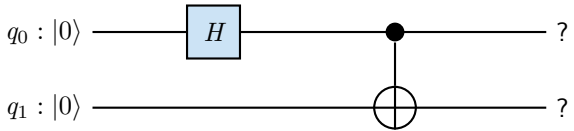
CNOT의 4×4 유니터리 행렬 표현

$$CX = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} = \begin{pmatrix} I_2 & 0 \\ 0 & X \end{pmatrix}$$

- $q_0 = 0$일 때(위쪽 2×2 블록): 항등행렬 I_2가 작용하여 상태를 유지합니다.
- $q_0 = 1$일 때(아래쪽 2×2 블록): Pauli-X 행렬이 작용하여 비트를 반전시킵니다.

양자 얽힘(Entanglement) 만들기: Bell 회로 개요

이제 H 게이트와 CNOT 게이트를 결합하여, 양자 알고리즘 계산 이점의 핵심 기반인 **얽힘 (Bell 상태)**을 직접 만듭니다:



워크시트 3 목표

초기 상태 $|00\rangle$에서 출발하여, 중간 상태 $|\psi_1\rangle$과 최종 얽힘 상태 $|\Phi^+\rangle$를 단계별로 직접 손계산합니다.

[손계산 1단계] 초기 상태와 1번 큐비트 Hadamard 작용

초기 상태의 텐서곱 표현

$$|\psi_0\rangle = |0\rangle \otimes |0\rangle = \begin{pmatrix}1\\0\end{pmatrix} \otimes \begin{pmatrix}1\\0\end{pmatrix} = \begin{pmatrix}1\\0\\0\\0\end{pmatrix} = |00\rangle$$

1단계: q_0에 Hadamard (H) 게이트 적용

$$|\psi_1\rangle = (H \otimes I)|\psi_0\rangle = (H|0\rangle) \otimes (I|0\rangle)$$

$$= \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}}\right) \otimes |0\rangle = \frac{1}{\sqrt{2}}(|0\rangle \otimes |0\rangle) + \frac{1}{\sqrt{2}}(|1\rangle \otimes |0\rangle)$$

$$\Longrightarrow \quad |\psi_1\rangle = \frac{|00\rangle + |10\rangle}{\sqrt{2}} = \frac{1}{\sqrt{2}}\begin{pmatrix}1\\0\\1\\0\end{pmatrix}$$

[손계산 2단계] CNOT 게이트 통과와 조건부 반전

중간 상태 $|\psi_1\rangle$에 CNOT 게이트 작용

$$|\psi_2\rangle = CX|\psi_1\rangle = CX\left(\frac{|00\rangle + |10\rangle}{\sqrt{2}}\right) = \frac{1}{\sqrt{2}}CX|00\rangle + \frac{1}{\sqrt{2}}CX|10\rangle$$

선형성에 따른 각 항별 CNOT 연산 결과

- 첫 번째 항 $CX|00\rangle$: 제어 큐비트가 0이므로 $\implies |00\rangle$ 유지!
- 두 번째 항 $CX|10\rangle$: 제어 큐비트가 1이므로 타깃 비트가 $0 \to 1$ 반전 $\implies |11\rangle$로 변환!

$$\implies |\psi_2\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}} = |\Phi^+\rangle$$

[손계산 3단계] 4×4 행렬곱으로 수치 재검증

행렬과 열벡터의 직접 곱셈으로 동일한 결과가 도출되는지 확인합니다:

행렬-벡터 곱셈 전개

$$|\psi_2\rangle = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 1/\sqrt{2} \\ 0 \\ 1/\sqrt{2} \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \cdot (1/\sqrt{2}) + 0 + 0 + 0 \\ 0 + 1 \cdot 0 + 0 + 0 \\ 0 + 0 + 0 \cdot (1/\sqrt{2}) + 1 \cdot 0 \\ 0 + 0 + 1 \cdot (1/\sqrt{2}) + 0 \end{pmatrix} = \begin{pmatrix} 1/\sqrt{2} \\ 0 \\ 0 \\ 1/\sqrt{2} \end{pmatrix}$$

Dirac 표기법 전개와 4×4 행렬 연산이 완벽히 일치합니다!

[손계산 검증] Bell 상태의 측정 확률 분석

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}|00\rangle + 0|01\rangle + 0|10\rangle + \frac{1}{\sqrt{2}}|11\rangle$$

Born 규칙에 따른 4가지 측정 확률 계산

- $P(00) = \left|\frac{1}{\sqrt{2}}\right|^2 = \frac{1}{2} = 50\%$
- $P(01) = |0|^2 = 0\%$ (절대 관측되지 않음)
- $P(10) = |0|^2 = 0\%$ (절대 관측되지 않음)
- $P(11) = \left|\frac{1}{\sqrt{2}}\right|^2 = \frac{1}{2} = 50\%$

놀라운 물리적 결과

두 큐비트를 동시에 측정하면, 서로 다른 값('01', '10')은 전혀 나오지 않고
오직 같은 값('00' 또는 '11')만 각각 50% 확률로 완벽하게 일치하여 관측됩니다!

[손계산 증명] 왜 얽힘 상태는 분리될 수 없는가? (비분리성 증명)

귀류법 증명: $|\Phi^+\rangle$가 독립된 두 상태의 텐서곱이라 가정

$$|\Phi^+\rangle = (a|0\rangle + b|1\rangle) \otimes (c|0\rangle + d|1\rangle) = ac|00\rangle + ad|01\rangle + bc|10\rangle + bd|11\rangle$$

계수 비교를 통한 모순 도출

1. $ac = 1/\sqrt{2} \implies a \neq 0$ 이고 $c \neq 0$
2. $ad = 0 \implies a \neq 0$ 이므로 반드시 $d = 0$
3. $bc = 0 \implies c \neq 0$ 이므로 반드시 $b = 0$
4. 그런데 네 번째 항 $bd = 1/\sqrt{2}$이어야 하는데, $b = 0, d = 0$이므로 $bd = 0 \neq 1/\sqrt{2}$!
   (모순!)

결론: Bell 상태 $|\Phi^+\rangle$는 단일 큐비트 상태들의 곱으로 절대 쪼갤 수 없는 '얽힘(Entangled)' 상태입니다!

1부 마무리 점검 및 중간 휴식 안내 (20분)

1부 핵심 달성도 체크

- ✓ 고전 50비트와 큐비트 50개의 차이 (2^n 차원 힐베르트 공간의 지수적 폭발)
- ✓ 텐서곱($\otimes$)과 혼합곱 공식 $(A \otimes B)(C \otimes D) = (AC) \otimes (BD)$
- ✓ 제어-NOT(CNOT) 게이트의 조건부 반전 규칙과 4×4 행렬 작용
- ✓ [손계산 워크시트] Bell 상태 $\frac{|00\rangle + |11\rangle}{\sqrt{2}}$ 유도 및 비분리성 증명 완료

20분간 휴식 및 2부 NumPy 실습 준비

- 휴식 후 2부에서는 **NumPy를 활용한 다중 큐비트 행렬 연산 및 얽힘 시뮬레이션(75분)**이 진행됩니다.
- Jupyter Notebook에서 실습 파일(`day2_numpy_entanglement.ipynb`)을 열어주세요.

2부 실습 개요: 선형대수학을 코드로 정복하기

"손으로 유도한 수학을 파이썬 배열과 행렬곱으로 100% 재현한다"

2부 인터랙티브 실습 로드맵 (75분)

1. **[컴퓨팅 원리]** 얽힘과 양자 우위: 얽힘이 없으면 50큐비트도 스마트폰 1초 컷!
2. **[알고리즘 연계]** 왜 Bell 상태인가? 다중 큐비트 알고리즘의 최소 연산 단위
3. **[Lab 1]** 'np.kron'을 이용한 텐서곱과 혼합곱 성질 수치 검증
4. **[Lab 2]** 4×4 CNOT 행렬곱 연산과 손계산 결과 1:1 대조 ('assert')
5. **[Lab 3]** 선형대수학적 얽힘 판별식 ($c_{00}c_{11} - c_{01}c_{10} \neq 0$) 함수 구현
6. **[2부 챌린지]** 4대 Bell 상태 ($|\Phi^{\pm}\rangle, |\Psi^{\pm}\rangle$) 전수 합성 및 직교성 검증
7. **[Lab 4]** QPU 내부의 부분 측정(Partial Measurement)과 조건부 상태 제어

얽힘의 컴퓨터공학적 본질: 양자 우위의 유일한 원천

만약 50큐비트 회로에 '얽힘'이 발생하지 않는다면?

모든 큐비트가 독립된 곱상태로 유지된다면:

$$|\psi\rangle = (\alpha_1|0\rangle + \beta_1|1\rangle) \otimes (\alpha_2|0\rangle + \beta_2|1\rangle) \otimes \cdots \otimes (\alpha_{50}|0\rangle + \beta_{50}|1\rangle)$$

- 50개 큐비트 상태를 저장하는 데 필요한 복소수는 2^{50}(1,125조 개)가 아닙니다.
- **오직** $2 \times 50 = 100$**개의 복소수**만 있으면 메모리에 전부 담깁니다!
- 즉, 스마트폰 메모리 1KB로도 0.001초 만에 완벽하게 시뮬레이션됩니다.

컴퓨터 관점에서의 결론

양자컴퓨터가 고전 슈퍼컴퓨터를 압도할 수 있는 유일한 원천은
"개별 상태로 쪼갤 수 없는 비분리 얽힘 상태(Entangled State)의 존재" 때문입니다!

다중 큐비트 알고리즘의 빌딩 블록: 왜 Bell 회로인가?

왜 우리는 Bell 회로를 가장 먼저 배우는가?

- 1큐비트 게이트(H, X, Z)만으로는 큐비트 간에 데이터를 주고받을 수 없습니다.
- 큐비트 간 상호작용(CNOT, CZ)이 결합하는 순간, **모든 양자 회로는 얽힘을 형성**합니다.
- 3일차에 배울 **Grover 알고리즘의 오라클과 확산 연산자** 역시 큐비트들을 정밀하게 얽어 파동을 간섭시킴으로써 정답 확률을 극대화합니다.

프로그래밍의 Hello World

프로그래밍 언어를 배울 때 가장 먼저 'printf("Hello World");'를 출력하듯,
Bell 회로는 모든 다중 큐비트 양자 알고리즘을 구성하는 가장 기초적인 연산 단위입니다.

[Lab 1-1] NumPy로 4차원 기저 벡터 정의하기

```python
import numpy as np

# 1. 단일큐비트기저정의
ket0 = np.array([[1.0], [0.0]], dtype=complex)
ket1 = np.array([[0.0], [1.0]], dtype=complex)

# 2. np.을kron 이용한차원 4 계산기저생성
ket00 = np.kron(ket0, ket0)  # |00> = [1, 0, 0, 0]^T
ket01 = np.kron(ket0, ket1)  # |01> = [0, 1, 0, 0]^T
ket10 = np.kron(ket1, ket0)  # |10> = [0, 0, 1, 0]^T
ket11 = np.kron(ket1, ket1)  # |11> = [0, 0, 0, 1]^T

print("|00> 기저벡터 :\n", ket00.flatten().real)
print("|11> 기저벡터 :\n", ket11.flatten().real)
assert np.allclose(ket00[0], 1.0) and np.allclose(ket11[3], 1.0)
```

[Lab 1-2] 게이트 텐서곱과 혼합곱 공식 검증

```
# 단일큐비트유니터리게이트
H = (1 / np.sqrt(2)) * np.array([[1, 1], [1, -1]], dtype=complex)
X = np.array([[0, 1], [1, 0]], dtype=complex)
Z = np.array([[1, 0], [0, -1]], dtype=complex)

# 혼합곱공식 : (A (x) B)(C (x) D) == (AC) (x) (BD)
# 좌변 (LHS): 4x4 행렬개를 2 각각만든후행렬곱
lhs = np.kron(H, X) @ np.kron(Z, H)

# 우변 (RHS): 2x2 행렬곱을먼저수행한후텐서곱
rhs = np.kron(H @ Z, X @ H)

# 수치검증
assert np.allclose(lhs, rhs), "혼합곱 공식이불일치합니다 !"
print("[Lab 1 성공] (H (x) X)(Z (x) H) == (HZ) (x) (XH) 일치확인완료  !")
```

[Lab 2-1] 4×4 CNOT 행렬과 $H \otimes I$ 구성

```python
# 1. 4x4 CNOT 행렬정의
CNOT = np.array([
    [1, 0, 0, 0],
    [0, 1, 0, 0],
    [0, 0, 0, 1],
    [0, 0, 1, 0]
], dtype=complex)

# 2. 에만q0 를H 적용하는 4x4 연산자 H (x) I
I2 = np.eye(2, dtype=complex)
H_I = np.kron(H, I2)

# 유니터리성전수검증  : U^dag @ U == I_4
assert np.allclose(CNOT.conj().T @ CNOT, np.eye(4, dtype=complex))
assert np.allclose(H_I.conj().T @ H_I, np.eye(4, dtype=complex))
print("CNOT 및 H (x) I 연산자유니터리성검증통과   !")
```

[Lab 2-2] 행렬곱(@)으로 Bell 상태 생성 및 손계산 대조

```
# 1. 초기상태에  |00> H (x) I 작용
psi_step1 = H_I @ ket00      # (|00> + |10>) / sqrt(2)

# 2. CNOT 게이트통과  -> Bell 상태 |Phi+>
psi_bell = CNOT @ psi_step1  # (|00> + |11>) / sqrt(2)

# 3. 부1 손계산결과와오차   1e-15 이내 1:1 대조
expected_bell = (1.0 / np.sqrt(2.0)) * (ket00 + ket11)
assert np.allclose(psi_bell, expected_bell), "손계산 결과와수치가일치하지않습니다   !"

print("[Lab 2 성공] 부1 손계산 Bell 상태와 100% 일치!")
print("계산된 상태벡터:\n", psi_bell.flatten().real)
```

선형대수학적 얽힘 판별식 (비분리성 판별 조건)

임의의 2큐비트 상태 $|\psi\rangle = c_{00}|00\rangle + c_{01}|01\rangle + c_{10}|10\rangle + c_{11}|11\rangle$의 계수 행렬 $M = \begin{pmatrix} c_{00} & c_{01} \\ c_{10} & c_{11} \end{pmatrix}$:

행렬식(Determinant)과 분리 가능성

$$\det(M) = c_{00}\,c_{11} - c_{01}\,c_{10}$$

- $\det(M) = 0$: **곱상태** $\Longrightarrow$ $(\alpha|0\rangle + \beta|1\rangle) \otimes (\gamma|0\rangle + \delta|1\rangle)$로 인수분해 가능
- $\det(M) \neq 0$: **얽힘 상태** $\Longrightarrow$ 독립 상태의 곱으로 인수분해 절대 불가

Bell 상태의 행렬식 값

$|\Phi^+\rangle \implies c_{00} = 1/\sqrt{2}, c_{01} = 0, c_{10} = 0, c_{11} = 1/\sqrt{2}$

$$\det = (1/\sqrt{2})(1/\sqrt{2}) - 0 \cdot 0 = 1/2 \neq 0 \implies \text{최대 얽힘(Maximal Entanglement)!}$$

[Lab 3] 얽힘 판별식 함수 구현 및 수치 검증

```python
def is_entangled(state_4d, tol=1e-9):
    c = state_4d.flatten()
    det = c[0] * c[3] - c[1] * c[2]
    entangled = not np.isclose(abs(det), 0.0, atol=tol)
    return entangled, float(abs(det))

# 1. Step 1 상태 |+0> 판별: 곱상태이므로 det == 이어야0 함
ent1, det1 = is_entangled(psi_step1)
assert not ent1 and det1 == 0.0, "Step 1 상태는분리가능한곱상태입니다   !"

# 2. 최종 Bell 상태 |Phi+> 판별: 얽힘상태이므로  det == 이어야0.5 함
ent2, det2 = is_entangled(psi_bell)
assert ent2 and np.isclose(det2, 0.5), "Bell 상태는최대얽힘상태입니다   !"

print(f"[Lab 3 성공] 곱상태 det={det1:.1f} 분리( 가능) / Bell 상태 det={det2:.3f} 최대( 얽
    힘)")
```

4대 Bell 상태 (Bell Basis)의 전체 체계

초기 입력 4가지 기저에 동일한 회로 $CNOT \cdot (H \otimes I)$를 통과시키면 4대 Bell 기저가 완성됩니다:

입력 기저	중간 상태 ($\|\psi_1\rangle$)	최종 Bell 상태 ($\|\psi_2\rangle$)	기호
$\|00\rangle$	$\frac{\|00\rangle+\|10\rangle}{\sqrt{2}}$	$\frac{\|00\rangle+\|11\rangle}{\sqrt{2}}$	$\|\Phi^+\rangle$
$\|01\rangle$	$\frac{\|01\rangle+\|11\rangle}{\sqrt{2}}$	$\frac{\|01\rangle+\|10\rangle}{\sqrt{2}}$	$\|\Psi^+\rangle$
$\|10\rangle$	$\frac{\|00\rangle-\|10\rangle}{\sqrt{2}}$	$\frac{\|00\rangle-\|11\rangle}{\sqrt{2}}$	$\|\Phi^-\rangle$
$\|11\rangle$	$\frac{\|01\rangle-\|11\rangle}{\sqrt{2}}$	$\frac{\|01\rangle-\|10\rangle}{\sqrt{2}}$	$\|\Psi^-\rangle$

4대 Bell 기저의 직교성 (Orthogonality)

이 4개의 얽힘 상태들은 4차원 복소 공간을 이루는 **새로운 정규직교 기저(Orthonormal Basis)**가 됩니다.

[2부 챌린지] NumPy로 4대 Bell 상태 전수 합성 및 직교성 증명

```
inputs = {"|00>": ket00, "|01>": ket01, "|10>": ket10, "|11>": ket11}
bell = {name: CNOT @ (H_I @ vec) for name, vec in inputs.items()}

# 대4 Bell 상태: |Phi+>, |Psi+>, |Phi->, |Psi->
phi_p, psi_p = bell["|00>"], bell["|01>"]
phi_m, psi_m = bell["|10>"], bell["|11>"]

# 개4 상태의상호직교성전수검증     : <u|v> == 0
assert np.isclose(np.vdot(phi_p, phi_m), 0.0)
assert np.isclose(np.vdot(phi_p, psi_p), 0.0)
assert np.isclose(np.vdot(phi_p, psi_m), 0.0)
assert np.isclose(np.vdot(psi_p, phi_m), 0.0)
assert np.isclose(np.vdot(psi_p, psi_m), 0.0)
assert np.isclose(np.vdot(phi_m, psi_m), 0.0)
print("부[2 챌린지성공 ] 대4 Bell 상태가상호직교기저임을전수증명완료       !")
```

QPU 내부의 부분 측정: 양자 오류 정정(QEC)과 조건부 제어

원격 통신이 아니라, **실제 양자 프로세서(QPU) 칩 내부**에서 부분 측정이 왜 필수적일까요?

양자 계산의 딜레마와 해결책

- 계산 도중 메인 큐비트를 직접 측정하면 계산 중이던 중첩과 얽힘이 파괴됩니다.
- **해결책**: 연산 큐비트 옆에 보조 큐비트(Ancilla)를 얽어놓고, **보조 큐비트만 부분 측정**합니다!
- 메인 데이터의 중첩은 유지한 채 오류 발생 여부(신드롬)만 감지하는 양자 오류 정정(QEC)의 기본 원리입니다.

수학적 사영 연산자(Projector)

$$P_0 = |0\rangle\langle 0| \otimes I, \quad P_1 = |1\rangle\langle 1| \otimes I \implies |\psi_{q1|k}\rangle = \frac{P_k|\Phi^+\rangle}{\sqrt{p_k}}$$

[Lab 4] 부분 측정 시뮬레이션 및 조건부 붕괴 함수 구현

```python
def measure_q0(state_4d):
    v = state_4d.flatten()
    p0 = float(abs(v[0])**2 + abs(v[1])**2)  # q0일=0 확률 (|00>, |01>)
    p1 = float(abs(v[2])**2 + abs(v[3])**2)  # q0일=1 확률 (|10>, |11>)
    state_q1_0 = np.array([v[0], v[1]], dtype=complex) / np.sqrt(p0)
    state_q1_1 = np.array([v[2], v[3]], dtype=complex) / np.sqrt(p1)
    return (p0, state_q1_0), (p1, state_q1_1)

(p0, s0), (p1, s1) = measure_q0(psi_bell)
print(f"q0=0 관측확률( {p0*100:.0f}%): q1 상태는즉시  |0> ->", s0.real)
print(f"q0=1 관측확률( {p1*100:.0f}%): q1 상태는즉시  |1> ->", s1.real)
assert np.allclose(s0, [1, 0]) and np.allclose(s1, [0, 1])
print("[Lab 4 성공] QPU 내부의조건부상태붕괴메커니즘검증완료       !")
```

2일차 총정리: 다중 큐비트와 얽힘의 핵심 원리

오늘 정리한 3대 컴퓨터공학적 원리

1. **상태공간의 지수적 확장 (2^n)**: 고전 비트 n개의 상태는 n개지만, 큐비트 n개의 복합 상태는 2^n차원 복소 힐베르트 공간의 단위벡터로 기술됩니다.
2. **2큐비트 유니터리 연산: CNOT**: 제어 큐비트 상태에 따라 타깃 큐비트에 X를 조건부 적용하여 큐비트 간 상호작용을 형성합니다.
3. **양자 얽힘: 양자 우위의 유일한 계산 자원**: 얽힘이 없다면 50큐비트도 스마트폰 1KB로 시뮬레이션됩니다. 비분리 얽힘 상태($c_{00}c_{11} - c_{01}c_{10} \neq 0$)가 존재해야 비로소 고전 슈퍼컴퓨터를 압도하는 진정한 양자 계산 가속이 발생합니다.

3일차 예고: Qiskit 회로 실습과 Grover 알고리즘

3일차부터는 글로벌 표준 프레임워크인 **Qiskit**을 도입하여 양자 회로를 직접 빌드하고, 위상 간섭을 이용해 비정렬 검색 속도를 획기적으로 높이는 **Grover 알고리즘**을 탐구합니다.